Network and Information Security (CT-486)

Complex Computing Problem [CLO3]

# Design and Break a Custom Cipher Based on Classical Techniques

**Course Code:** CT-486 | **Total Marks:** 10 | **Max Group Members:** 04 | **Submission Deadline:** 8th November 2026

## 1. Combination of Classical Techniques to Form the Custom Cipher

Per the project specification, the custom cipher is designed as a two-stage hybrid product cipher combining two distinct classical techniques studied in class:

- **Stage 1: Polyalphabetic Substitution (Vigenère Cipher):** Periodic key shifting using a secret keyword of length ≥ 10 characters (e.g., CRYPTOGRAPHYKEY, length 15). It achieves Claude Shannon's principle of *Confusion* by breaking monoalphabetic letter frequencies. Each character is shifted: C_1[i] = (P[i] + K_V[i mod L_V]) mod 26.
- **Stage 2: Polygraphic Block Transformation (Hill Cipher):** Linear algebraic 2x2 matrix multiplication over ■$_{26}$ using an invertible key matrix K_H derived from a key phrase of length ≥ 10 characters (e.g., HILLCIPHERKEY, length 13). It achieves *Diffusion* by mixing adjacent letter pairs (x1, x2) into (y1, y2) such that gcd(det(K_H), 26) = 1.

```
Plaintext (P) ■■> [Stage 1: Vigenere Substitution] ■■> Intermediate (C_1) ■■> [Stage 2: Hill Block Matrix] ■■> Ciphertext (C) Key: K_V (length >= 10) Key: K_H (2x2 Matrix, phrase >= 10)
```

## 2. Algorithms for Encryption and Decryption

### 2.1 Encryption Algorithm

```
Algorithm 1: Custom Cipher Encryption Input : Plaintext P, Vigenere Key K_V (length >= 10), Hill Key Matrix K_H (2x2) Output: Ciphertext C 1. P_clean <- Filter uppercase A-Z characters from P 2. If Length(P_clean) mod 2 != 0 Then P_clean <- P_clean + 'X' // Pad for 2x1 blocks 3. N <- Length(P_clean), Initialize array C_1 of length N 4. For i <- 0 to N - 1 Do 5. shift <- Ord(K_V[i mod Length(K_V)]) - 65 6. C_1[i] <- (Ord(P_clean[i]) - 65 + shift) mod 26 7. End For 8. Initialize array C_nums of length N 9. For j <- 0 to (N / 2) - 1 Do 10. x1 <- C_1[2*j], x2 <- C_1[2*j + 1] 11. C_nums[2*j] <- (K_H.a * x1 + K_H.b * x2) mod 26 12. C_nums[2*j + 1] <- (K_H.c * x1 + K_H.d * x2) mod 26 13. End For 14. Return string of characters corresponding to C_nums
```

### 2.2 Decryption Algorithm

```
Algorithm 2: Custom Cipher Decryption Input : Ciphertext C, Vigenere Key K_V, Hill Key Matrix K_H Output: Original Plaintext P 1. C_nums <- Convert uppercase A-Z characters of C to integers (0-25), N <- Length(C_nums) 2. Compute K_inv <- (det(K_H))^-1 * [[d, -b], [-c, a]] mod 26 3. Initialize array C_1 of length N 4. For j <- 0 to (N / 2) - 1 Do 5. y1 <- C_nums[2*j], y2 <- C_nums[2*j + 1] 6. C_1[2*j] <- (K_inv.a * y1 + K_inv.b * y2) mod 26 7. C_1[2*j + 1] <- (K_inv.c * y1 + K_inv.d * y2) mod 26 8. End For 9. Initialize array P_chars of length N 10. For i <- 0 to N - 1 Do 11. shift <- Ord(K_V[i mod Length(K_V)]) - 65 12. P_chars[i] <- Chr(((C_1[i] - shift + 26) mod 26) + 65) 13. End For 14. Return concatenated string of P_chars
```

## 3. Security Analysis of the Custom Cipher

**Confusion & Diffusion Assessment:** The Vigenère stage prevents direct single-letter frequency leakage (such as English letter 'E' at 12.7%). Simultaneously, the Hill stage spreads the statistical profile across adjacent character pairs. Modifying a single character alters two output symbols, ensuring strong diffusion.

**Index of Coincidence (IoC) Cryptanalysis:** The Index of Coincidence measures the probability that two randomly selected letters from the text are identical: IC = Σ f_i(f_i - 1) / (N(N - 1)).

| Text / Cipher Sample | Observed Index of Coincidence (IC) | Vulnerability Status |
|---|---|---|
| Standard English Plaintext | 0.0667 (Expected) | Natural language baseline |
| Single-Stage Shift (Caesar) Cipher | 0.0655 | Highly Vulnerable (Exact frequency preserved) |
| Uniform Random / Polyalphabetic Target | 0.0385 | Theoretical ideal random spread |
| **Custom Hybrid Cipher (Vigenère + Hill)** | **0.0405** | **Resistant to monographic frequency attacks** |

## 4. Description of Attack Methods and Results

**Threat Model:** An adversary intercepts full ciphertext C and possesses a known plaintext prefix (first 36 characters) from standard application headers. The goal is to recover both keys and decrypt the entire remaining ciphertext.

**Attack Algorithm:** The search space of 2x2 invertible matrices modulo 26 is $|GL(2, \blacksquare_{26})| = 157,248$. The attacker iterates through candidate matrices, partially inverting the Hill stage on the known prefix: [x1, x2]^T = K_H^-1 * [y1, y2]^T mod 26. The candidate Vigenère key stream K[i] = (x_i - P_known[i]) mod 26 must strictly satisfy periodicity K[i] == K[i + L_V]. Inconsistent matrices are pruned instantly, isolating the exact Hill matrix and Vigenère key in milliseconds.

| Ciphertext Length (N) | Known Chars (M) | Success Rate | Avg Attack Time (ms) | Candidate Matrices Evaluated |
|---|---|---|---|---|
| 100 chars | 36 | 100.0% | 83.1 ms | 42,881 |
| 250 chars | 36 | 100.0% | 83.1 ms | 42,881 |
| 500 chars | 36 | 100.0% | 84.2 ms | 42,881 |
| 1,000 chars | 36 | 100.0% | 84.0 ms | 42,881 |

## 5. Performance Metrics and Efficiency Considerations

**Asymptotic Time Complexity:**

• **Custom Hybrid Encryption/Decryption:** O(N) linear time (Stage 1 takes O(N) operations; Stage 2 processes N/2 blocks of size 2x2 taking O(1) per block, total O(N)).

• **Baseline Shift Cipher:** O(N) linear time (single modular shift per character).

| Message Length (N) | Custom Enc (ms) | Custom Dec (ms) | Shift Enc (ms) | Shift Dec (ms) | Overhead Factor |
|---|---|---|---|---|---|
| 100 chars | 0.0300 ms | 0.0300 ms | 0.0126 ms | 0.0126 ms | 2.38x |
| 500 chars | 0.1426 ms | 0.1486 ms | 0.0601 ms | 0.0611 ms | 2.40x |
| 1,000 chars | 0.2831 ms | 0.2940 ms | 0.1183 ms | 0.1198 ms | 2.45x |

| | | | | | |
|---|---|---|---|---|---|
| 5,000 chars | 1.3792 ms | 1.4353 ms | 0.5583 ms | 0.5682 ms | 2.50x |
| 10,000 chars | 2.8358 ms | 2.9351 ms | 1.1191 ms | 1.1346 ms | 2.55x |

## 6. Suggestions for Improving the Cipher's Security

1. **Autokey Key Scheduling:** Replace the periodic repeating Vigenère keyword with an Autokey stream (where subsequent key characters are drawn from plaintext: K[i] = P[i - L_V]). This eliminates the periodic alignment exploited by the attack while preserving strict O(N) linear runtime.

2. **Higher-Dimensional Matrix (3x3 or 4x4):** Expanding block size to m=3 or m=4 scales the invertible matrix key space to $1.6 \times 10^{12}$ and $8.6 \times 10^{21}$, rendering exhaustive matrix evaluation completely intractable with negligible linear overhead.

3. **Non-Linear S-Box Permutation:** Interposing a non-linear substitution stage between Vigenère and Hill disrupts linear modular congruences over $\blacksquare_{26}$.

## 7. Code Submission & Execution Details

All project source files are hosted in the public GitHub repository:
**Repository Link:** https://github.com/ameemakhtr/NIS_CCP

**Repository Contents:**
• custom_cipher.py: Hybrid cipher implementation & baseline Shift cipher.
• attack.py: Frequency analyzer & Known-Plaintext Attack solver.
• benchmark.py: Latency & throughput benchmarking suite.
• test_suite.py: Automated unit tests.
• live_test.py: Standalone live demonstration script.
• main.py: Interactive CLI application.

**Execution Command:** python main.py --demo